Topic: Stack using Singly Linked List

Q. Explain how a Stack can be implemented using a Singly Linked List. Provide the logic for the Push and Pop operations.

> 📌 **Definition to Remember**
> A Stack follows the **LIFO (Last In, First Out)** principle. Implementing it using a **Singly Linked List** allows the stack to grow dynamically, preventing Stack Overflow (unless heap memory runs out). The **Head** of the linked list acts as the **Top** of the stack.

---

1. Conceptual Mapping: Linked List to Stack

To maintain O(1) time complexity for both Push and Pop operations, all insertions and deletions must occur at the **beginning (head)** of the linked list.

Stack Operation	Linked List Equivalent
Top Pointer	`head` pointer
Push(value)	Insert node at the **beginning**
Pop()	Delete node from the **beginning**

2. Push Operation (Insert at Beginning)

Adds a new item to the top of the stack.

Algorithm:

```
1. Allocate memory for new_node.
2. If allocation fails, print "Stack Overflow" (Heap full).
3. new_node→data = value
4. new_node→next = top    // Point new node to current top
5. top = new_node         // Update top to the new node
```

3. Pop Operation (Delete from Beginning)

Removes and returns the item from the top of the stack.

Algorithm:

```
1. If top == NULL, print "Stack Underflow" (Empty stack).
2. temp = top               // Temporarily hold the top node
3. value = temp→data        // Extract data
4. top = top→next           // Move top to the next node
5. free(temp)               // Free memory
6. return value
```

4. Advantages of Linked List Implementation

- **Dynamic Size:** The stack can grow and shrink dynamically. It is limited only by system memory, unlike an array which has a fixed pre-defined size.

- **No Memory Wastage:** Memory is allocated precisely when a new element is pushed.
- **Efficiency:** Both Push and Pop operate in constant time **O(1)**.

> ⭐ **Must-Write Points (for 10 marks)**
> 1. Stack is LIFO. Linked List implementation makes the stack **dynamic in size**.
> 2. Prevents "Stack Overflow" related to fixed array limits (only fails if heap memory is exhausted).
> 3. The **Head** of the linked list represents the **Top** of the stack.
> 4. **Push = Insert at Beginning**. Time complexity is O(1).
> 5. **Pop = Delete from Beginning**. Time complexity is O(1).
> 6. Push Logic: `new_node→next = top; top = new_node;`
> 7. Pop Logic: `temp = top; top = top→next; free(temp);`

> ⚡ **Quick Recall**
> `Stack via Linked List → Dynamic Size (no array limits) → Head = Top → Push = Insert at Head (O(1)) → Pop = Delete from Head (O(1))`